

Executive Summary

Alche Studio's homepage is a **static site** built with the [Astro](#) framework (v5.8.1) and modern web technologies ¹. It uses **TypeScript/JavaScript** for interactivity (via React components) and **SCSS modules** for styling ² ³. A WebGL scene (Three.js v0.177.0 ¹) and **GSAP (v3.13)** animations (including ScrollTrigger) are used for dynamic effects ⁴ ¹. Assets include PNG and SVG images (e.g. `/top/fortnite.png`, `/top/ue2.png`, `/common/alche_logo.svg`), Japanese/Latin webfonts (e.g. *Bodoni Moda*, *Noto Sans JP* ⁵ ⁶), and various logos. The site links to external services (blog.note, X/Twitter, YouTube, Notion, etc.) but otherwise loads no dynamic backend (deployed as a static site). This report catalogs all detected technologies, file paths, and assets on the homepage, and provides a step-by-step guide to reproduce it locally.

Technical Stack & Libraries

- **Framework:** Astro v5.8.1 (static site generator) ¹, using React components for UI ¹. Astro's build uses Vite/Rollup under the hood.
- **Language:** TypeScript (code in `src/ts/`) and modern ECMAScript via Babel (e.g. `@babel/preset-react` for JSX) ⁷ ⁸.
- **3D/WebGL:** Three.js v0.177.0 for rendering ¹, plus a custom `ore-three` extension library ⁹. Shaders are written in GLSL and managed with glslify ¹⁰.
- **Animations:** GSAP (GreenSock v3.13) with ScrollTrigger for scroll-based animations ⁴. CSS animations/transitions may also be used (not explicitly listed).
- **Styling:** SASS/SCSS with CSS Modules per-component (Astro supports scoped styles) ² ³. No global CSS framework is evident.
- **Other libs:** `@0b5vr/tweakpane-plugin-rotation` (for parameter tweaking) and possibly swup (SPA-like page transitions via Astro swup integration) ¹¹. Numerous Babel and ESLint plugins are installed (per [LICENSE]) but not directly visible on the homepage.
- **Fonts:** The site uses variable Google Fonts via [Fontsource](#) packages – notably *Bodoni Moda* (variable) and *Noto Sans JP* ⁵ ⁶. These are likely loaded via CSS `@font-face`.
- **Icons:** No standard icon font is obvious. The navigation and footer use text or SVGs.
- **Image Formats:** PNG (e.g. `/top/fortnite.png`, `/top/ue2.png`), SVG (logo: `/common/alche_logo.svg`), and WebP/JPEG for project images elsewhere. All homepage assets are static.





Homepage image examples: The “Fortnite Creative Works” and “Unreal Engine Works” sections use PNG images (`fortnite.png` , `ue2.png`) stored under `/top/` ¹² ¹³ . The Alche logo is an SVG (`/common/alche_logo.svg`) ¹⁴ .

- **Paths:** Key paths on the homepage include:
 - `/top/fortnite.png` , `/top/ue2.png` (PNG banners) ¹⁵ ¹³
 - `/common/alche_logo.svg` (company logo) ¹⁴
 - Other `/common/` assets (CSS/JS likely under the hood, not textually visible).
- **Build Tools:** Node/NPM is used (Astro CLI). The license page notes `npm run build` and `npm run astro` commands ¹⁶ . Package manager: NPM or Yarn (Astro standard).
- **Bundlers/Transpilers:** Astro relies on Vite (Rollup) for bundling ¹⁷ . Babel is used for JSX/TSX via plugins ⁸ . TypeScript is compiled by esbuild (via Astro) as indicated by the `ts/` source layout ⁷ .
- **Server/CDN:** No server-side code is visible; the site is likely deployed as a static site (Astro-generated) on a CDN/hosting service. (Domain `alche.studio` is SSL-enabled, but hosting provider isn’t disclosed.)
- **Third-Party Services:** The homepage links to Tech Blog (`blog.alche.studio`), `note.com`, X (`twitter.com`), YouTube, Notion (`alche.notion.site` contact form), and `career.alche.studio` for recruiting. News links point to external PR sites (`prtimes.jp`). No analytics or API calls are directly evident in the HTML.
- **Network Requests:** All page assets (CSS, JS, images) appear local to `alche.studio`. External requests are limited to linked resources (fonts may load from Google or Fontsource via NPM bundling).

Asset Inventory

- **Images:** See table below for key assets:

File Name	Path	Type	Purpose
<code>fortnite.png</code>	<code>/top/fortnite.png</code>	PNG	“Fortnite Creative Works” banner ¹²
<code>ue2.png</code>	<code>/top/ue2.png</code>	PNG	“Unreal Engine Works” banner ¹³
<code>alche_logo.svg</code>	<code>/common/alche_logo.svg</code>	SVG	Alche company logo ¹⁴
News date images	(none visible)	-	Dates shown as text (no img)

File Name	Path	Type	Purpose
Various project images (e.g. in Works pages)	(under <code>/common/</code> or works folders)	JPG/ WebP?	Project screenshots (in works detail pages, not on homepage)
- Fonts: <i>Bodoni Moda</i> (variable weight) and <i>Noto Sans JP</i> (variable) ⁵ ⁶ . These likely reside under <code>/node_modules/@fontsource/...</code> and are included in the build.			
- Icons: No icon font identified. The navigation uses text and possibly SVG arrow icons (e.g. the “Top” anchor may have an SVG, but [browser.open] did not reveal it).			
- CSS/JS Files: The raw HTML (not seen by our tools) probably references compiled CSS/JS bundles. Common practice: Astro places CSS/JS in <code>/_astro/</code> or <code>/dist/</code>. We did not extract exact filenames. However, given the license page mentions ESBuild and Rollup plugins, expect compiled assets (hashed) in <code>dist</code>.			
- Media/Anim Assets: Aside from images, the homepage uses no video or Lottie files. The “scroll to explore” hint suggests scroll-triggered animations (via GSAP) rather than media.			

Animation & Interactivity

- **Scroll Animations:** GSAP's ScrollTrigger is specifically cited ¹⁸, so sections likely animate into view. For example, elements with classes like `works_intro`, `vision`, etc., probably fade or slide as the user scrolls.
- **WebGL Scene:** The presence of Three.js, GLSL and custom `ore-three` implies a 3D scene is rendered. It might be the hero background or a canvas in the Hero section (“Architect worlds that move hearts...”). The license (code structure) suggests a `TopPageMainScene`. We did not capture the canvas, but the code exists (per license mention of `TopPageMainScene` ¹⁹).
- **No Lottie or other animation libraries** were noted. The stack suggests **GSAP + Three.js** handle all dynamics. CSS transitions may supplement small effects.

Reproduction Guide

To clone the homepage locally, use Astro with React and the above libraries. Steps:

```
# 1. Initialize a new Astro project
npm create astro@latest my-alche-clone
cd my-alche-clone

# 2. Install needed packages (Astro v5.8.1, React, Three.js, GSAP, etc.)
npm install astro@5.8.1 react react-dom three@0.177.0 gsap @fontsource-variable/bodoni-moda @fontsource-variable/noto-sans-jp glslify
```

Folder structure:

```
my-alche-clone/
├─ public/                # Static assets (served at root)
│   └─ top/
│       └─ fortnite.png    # Fortnite image (from alche.studio)
│       └─ ue2.png          # Unreal image
│       └─ common/
│           └─ alche_logo.svg # Alche logo
├─ src/
│   └─ components/          # Astro/React UI components
│       └─ Header.astro
│       └─ HeroSection.astro
│       └─ WorksList.astro
│       └─ Footer.astro
│   └─ layouts/             # (Optional: common layout)
│       └─ pages/
│           └─ index.astro    # Homepage content
└─ astro.config.mjs         # Astro config (if needed for integrations)
```

In `src/pages/index.astro`, layout the sections as in the original: header/nav, Hero, News, Works summary, Mission/Vision, Services. Use React components or plain Astro depending on preference. For example, `HeroSection.astro` might contain the `<canvas>` for Three.js and overlay text.

Hero Three.js snippet (React component):

```
// src/components/TopScene.jsx
import { useEffect, useRef } from 'react';
import * as THREE from 'three';
import { OreThree } from 'ore-three'; // (assuming this custom lib is available)
import { GLTFLoader } from 'three/examples/jsm/loaders/GLTFLoader';

export default function TopScene() {
  const mountRef = useRef();
  useEffect(() => {
    const scene = new THREE.Scene();
    const camera = new THREE.PerspectiveCamera(75,
      mountRef.current.clientWidth/mountRef.current.clientHeight, 0.1, 1000);
```

```

    const renderer = new THREE.WebGLRenderer({ antialias: true, alpha:
true });
    renderer.setSize(mountRef.current.clientWidth,
mountRef.current.clientHeight);
    mountRef.current.appendChild(renderer.domElement);

    // Example: add a simple cube or custom scene
    const geometry = new THREE.BoxGeometry();
    const material = new THREE.MeshBasicMaterial({ color: 0x156289,
wireframe: true });
    const cube = new THREE.Mesh(geometry, material);
    scene.add(cube);
    camera.position.z = 5;

    function animate() {
        requestAnimationFrame(animate);
        cube.rotation.x += 0.01;
        cube.rotation.y += 0.01;
        renderer.render(scene, camera);
    }
    animate();

    // Cleanup on unmount
    return () => { renderer.dispose();
mountRef.current.removeChild(renderer.domElement); };
}, []);
    return <div ref={mountRef} class="hero-canvas" />;
}

```

Include this in the Hero section (`<TopScene/>`). Adjust as needed (the real site likely uses an imported GLTF model or complex scene via `ore-three`).

Animation with GSAP (in Hero or on scroll):

```

// e.g. in a React effect for animating intro text
import { useEffect } from 'react';
import { gsap } from 'gsap';
import { ScrollTrigger } from 'gsap/ScrollTrigger';
gsap.registerPlugin(ScrollTrigger);

export default function HeroText() {
    useEffect(() => {
        gsap.from(".hero-title", { opacity: 0, y: -50, duration: 1 });
        gsap.to(".mission-section", {
            scrollTrigger: { trigger: ".mission-section", start: "top 80%" },
            opacity: 1, y: 0, duration: 1
        });
    }, []);
}

```

```
    return <h1 class="hero-title">Architect worlds that move hearts...</h1>;
  }
```

This is illustrative; adjust selectors to match your HTML.

SCSS Modules: In Astro/React components, you might use CSS Modules, e.g.

`HeroSection.module.scss`. Enable SCSS in Astro config if needed.

Build and Run:

```
npm run dev      # start local dev server
npm run build    # production build
npm run preview  # preview built site
```

```

timeline
  title Reproduction Steps
  2026-05-05  Set up Astro project and install dependencies
  2026-05-05  Copy static assets (images, fonts) into `public/`
  2026-05-05  Create layout and components (Header, Hero, News, etc)
  2026-05-05  Implement animations (Three.js scene, GSAP scroll triggers)
  2026-05-05  Build and test local site

```

Library Versions Comparison: (from LICENSE and source)

Library/Framework	Detected Version	Role
Astro	5.8.1	Static site generator ¹
React	(integrated)	UI components ¹
Three.js	0.177.0	3D/WebGL rendering ¹
ore-three	custom	Three.js extension ⁹
GSAP	3.13.0	Animations (ScrollTrigger) ⁴
glslify / GLSL	N/A	Shader modules (WebGL) ¹⁰
SASS/SCSS Modules	N/A	CSS preprocessor ²
swup (Astro)	N/A	SPA-like page transitions (Astro integration) ¹¹

Each tool is cited from Alche's official LICENSE page or visible content ¹ ⁴ ² .

Assumptions/Notes: Any server-side logic is unspecified; we assume none (static). Exact CSS or component code wasn't directly extractable, so layout reproduction relies on visual and HTML clues (the text content above was obtained from the site). Undetectable items (like minor CSS frameworks or analytics) are assumed absent. If building exactly, one may need to inspect the original repo or use browser devtools to grab precise CSS/JS chunks.

Sources: All information is drawn from alche.studio pages and the site's open-source licenses (License page) ¹ ⁴ . Images are from the site's public directories ^{【4†】} ^{【5†】} . Any content not found on alche.studio (e.g. hosting details) is inferred and noted as assumption.

¹ ² ³ ⁴ ⁵ ⁶ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹⁶ ¹⁷ ¹⁸ ¹⁹ LICENSE | Alche, Inc

<https://alche.studio/license/>

¹² ¹³ ¹⁵ Alche, Inc

<https://alche.studio/>

¹⁴ alche.studio

https://alche.studio/common/alche_logo.svg